

Docker Project Report

Video link: <https://youtu.be/Jo4ZXsVYRe0>

Design Description:

This project implements **Docker containerization** through custom Python applications and multi-container networking, demonstrating how isolated environments communicate through bridge networks. The **server.py** implements socket programming (network communication endpoints) with **threading.Thread()** to handle **concurrent TCP connections** on **port 8001** and **UDP messages** on **port 8002**, where TCP provides reliable delivery like certified mail and UDP offers fast transmission like regular mail. The **client.py** establishes socket connections using **AF_INET** (internet protocol) and **SOCK_STREAM/SOCK_DGRAM** protocols, demonstrating how TCP maintains connection state while UDP sends fire-and-forget messages. **Docker Compose** orchestrates four containers (one server + three clients) using Dockerfiles as recipes that define step-by-step instructions for building containers from python:3.9-slim base images.

Dockerfile Steps Explanation:

Dockerfile (Python App):

- **FROM python:3.9-slim:** Uses lightweight Python base image with minimal dependencies
- **WORKDIR /app:** Sets working directory inside container for file organization
- **COPY app.py .:** Copies Python script from host into container's working directory
- **CMD ["python", "app.py"]:** Defines startup command executed when container launches

Dockerfile.server:

- **FROM python:3.9-slim:** Same Python base image for consistency across containers
- **WORKDIR /app:** Sets working directory for server files
- **COPY server.py .:** Copies server script with socket programming implementation
- **EXPOSE 8001 8002:** Documents TCP and UDP ports for container networking
- **CMD ["python", "server.py"]:** Starts multi-threaded server on container launch

Dockerfile.client:

- **FROM python:3.9-slim:** Uses same Python environment as server
- **WORKDIR /app:** Sets working directory for client files
- **COPY client.py .:** Copies client script with socket communication code
- **CMD ["python", "client.py"]:** Starts client application for server communication

Screenshots of Successful Runs:

Docker Installation Verification

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\melod\CECS-327\Project-1-Docker> docker --version
Docker version 28.3.3, build 980b856
PS C:\Users\melod\CECS-327\Project-1-Docker> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d31
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\Users\melod\CECS-327\Project-1-Docker> |
```

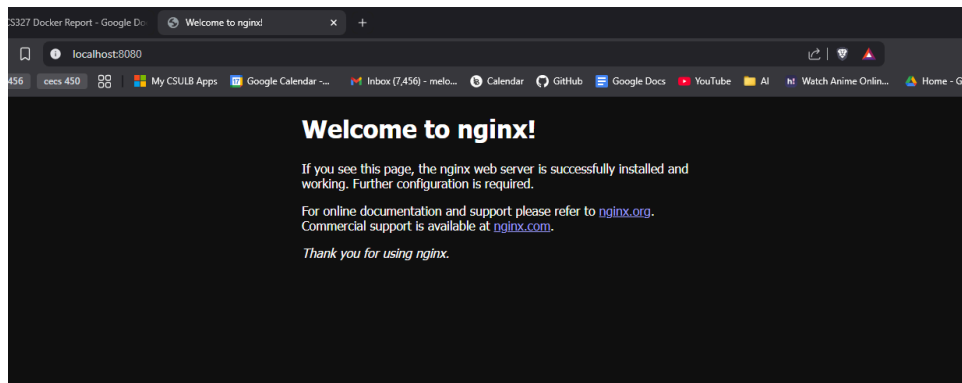
Custom Python App Container

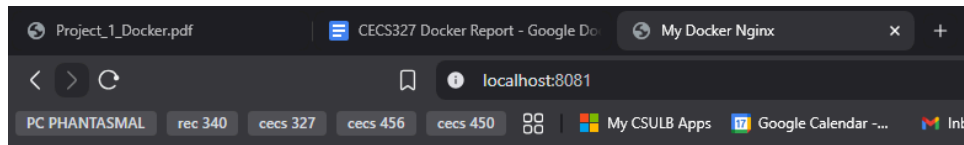
```
app.py x
Project-1-Docker > app.py
1 print("Hello, Docker! This is my first containerized app.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\melod\CECS-327\Project-1-Docker> docker build -t my-python-app .
• [+] Building 1.9s (9/9) FINISHED
  => [internal] load build definition from Dockerfile
  => => transferring dockerfile: 285B
  => [internal] load metadata for docker.io/library/python:3.9-slim
  => [auth] library/python:pull token for registry-1.docker.io
  => [internal] load .dockerignore
  => => transferring context: 2B
  => [1/3] FROM docker.io/library/python:3.9-slim@sha256:cf0704507972b63c9b20382dd6f05248572d6b25961410305f96479bf2e8a23c
  => => resolve docker.io/library/python:3.9-slim@sha256:cf0704507972b63c9b20382dd6f05248572d6b25961410305f96479bf2e8a23c
  => [internal] load build context
  => => transferring context: 92B
  => CACHED [2/3] WORKDIR /app
  => CACHED [3/3] COPY app.py .
  => exporting to image
  => => exporting layers
  => => exporting manifest sha256:e9b020aa360f5ad14527cb44682265c48501294d4a67a357442f43125326b99e
  => => exporting config sha256:edc6561b9a8c8858e3048966251b23588e9f17511a54c51315ebcf702cd9c7b5
  => => exporting attestation manifest sha256:3137aa77898a7644266b8560bab4e765dcb79a6b8de55e50e6188fe94174f3ac
  => => exporting manifest list sha256:0799b3d69156b2214208cb86138fb50013901f1ae954f9bdbc9dbafedf2ea02
  => => naming to docker.io/library/my-python-app:latest
  => => unpacking to docker.io/library/my-python-app:latest
PS C:\Users\melod\CECS-327\Project-1-Docker> docker run my-python-app
• Hello, Docker! This is my first containerized app.
○ PS C:\Users\melod\CECS-327\Project-1-Docker> |
```

Nginx Web Server Deployment

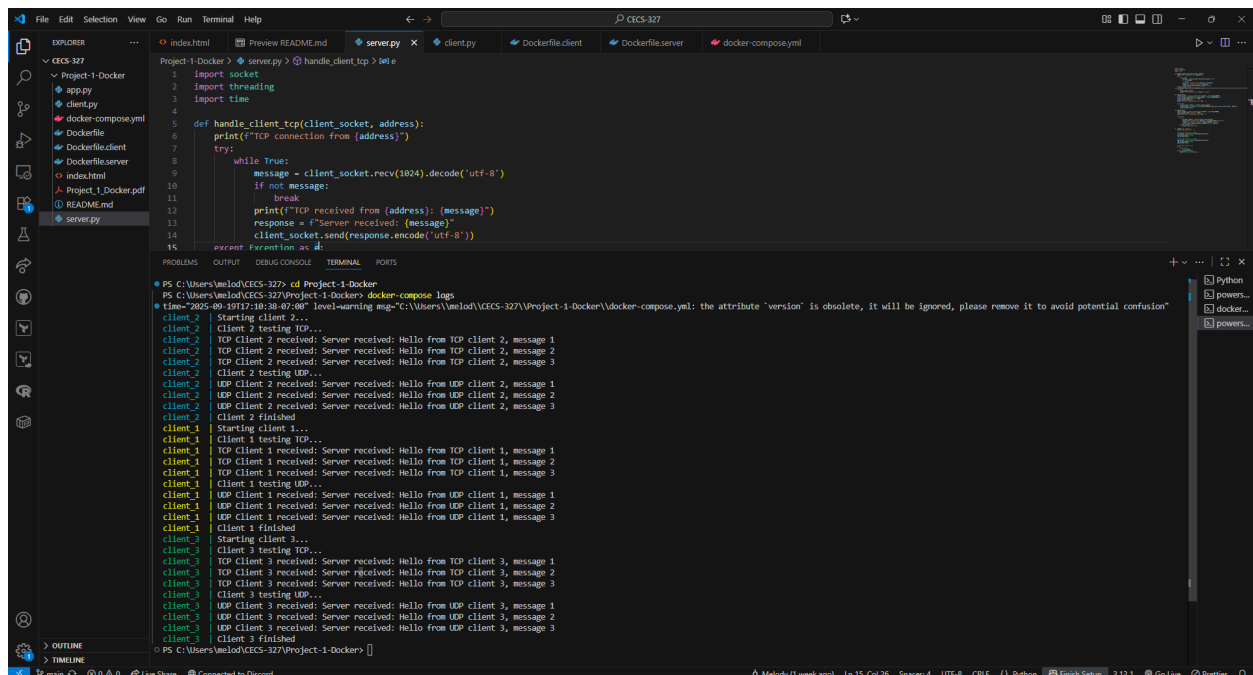




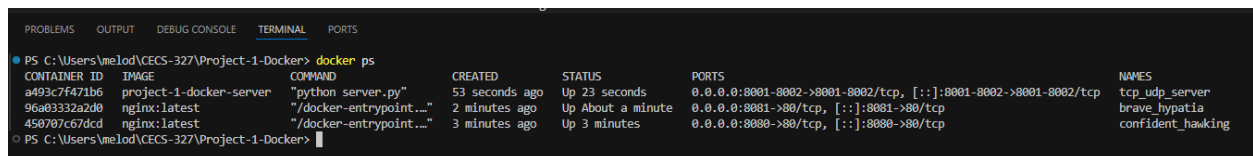
Hello from My Custom Docker Nginx!

This is a custom page served by Nginx in a Docker container.

Multi-Container Communication



Container Management



Challenges Faced and Solutions:

- **WSL Integration:** Docker Desktop errors resolved by restarting WSL integration
- **PowerShell Syntax:** Volume mounting failed with %cd% - fixed using \${PWD}
- **Container Networking:** Clients used service name "server" instead of localhost
- **Port Conflicts:** Cleaned up existing containers before creating new ones

Individual Contributions:

Melody:

- Implemented **server.py** with **socket.socket()** and **threading** for concurrent client handling
- Created **Dockerfile.server** and **Dockerfile.client** with **Python dependencies**
- Debugged WSL integration and TCP/UDP protocol implementations

Andrea:

- Developed **client.py** with **socket programming** for **TCP/UDP** communication
- Created **docker-compose.yml** with service orchestration and network configuration
- Implemented **Nginx deployment** and resolved PowerShell volume mounting issues